

Project Proposal

Enhancing xv6: CPU Time Accounting & Priority-Based Scheduling

Course	CSE323 — Operating Systems Section 3
Project Title	Enhancing xv6: CPU Time Accounting & Priority-Based Scheduling
Student 1	Akib Hassan Rifat ID: 2312379042
Student 2	Tauhid Ara Himu ID: 2312187042
Submission Date	March 08, 2026

1. Project Overview

This proposal describes two new features for the xv6 RISC-V operating system. Both are implemented via new system calls, are independent of each other, and address real OS functionality missing from the stock xv6 build. The goal is to gain practical experience with kernel internals specifically process management, scheduler design, and CPU time tracking.

2. Feature 1 — Per-Process CPU Time Accounting

2.1 Motivation

xv6 does not expose any CPU-usage information to user programs. There is no way to ask how much CPU time a process has consumed a fundamental observability feature present in every production OS (e.g., `getrusage()` on Linux). Adding it teaches how the kernel tracks time across context switches.

2.2 Design

Two fields will be added to the `proc` struct in `kernel/proc.h`: `cpu_ticks_total` (cumulative ticks the process has run) and `cpu_ticks_start` (tick value when the process was last scheduled in). The scheduler in `kernel/proc.c` will update these on every context switch. A new system call `getprocinfo(int pid, struct procinfo *buf)` will expose this data to user space.

2.3 New System Calls

- `int getprocinfo(int pid, struct procinfo *buf)` returns `pid`, `name`, `state`, and `cpu_ticks_total`. Returns 0 on success, -1 if `pid` not found.
- User tool `cpuusage` prints a live table of all running processes with their accumulated CPU ticks.

2.4 Testing Plan

- Run a CPU-bound loop; verify `cpu_ticks_total` grows proportionally.
- Verify a sleeping process does not accumulate ticks.
- Run `cpuusage` alongside multiple processes and confirm per-process totals are distinct.

3. Feature 2 — Priority-Based Process Scheduling

3.1 Motivation

xv6 uses a round-robin scheduler that gives every runnable process equal CPU time. This does not allow important tasks to run sooner than less critical ones. A priority scheduler lets the kernel always run the highest-priority process next, which is essential for understanding how real OSes manage CPU allocation.

3.2 Design

A priority field (`int priority`) will be added to the `proc` struct. Lower value means higher priority (e.g., priority 1 runs before priority 5). The scheduler in `kernel/proc.c` will be modified to scan all `RUNNABLE` processes and always select the one with the lowest priority value. Default priority for new processes will be 5.

3.3 New System Calls

- `int setpriority(int pid, int priority)` sets the priority of a process. Returns 0 on success, -1 if pid not found.
- `int getpriority(int pid)` returns the current priority value of a process, or -1 if not found.

3.4 Testing Plan

- Spawn multiple CPU-bound processes with different priorities; verify high-priority ones finish first.
- Call `setpriority()` dynamically and observe scheduler behaviour change in real time.
- Use `getpriority()` to confirm values are stored and retrieved correctly.

4. Project Timeline

Milestone	Feature 1: CPU Accounting	Feature 2: Priority Scheduling
Proposal (Mar 8)	Design <code>getprocinfo()</code> interface	Design priority scheduler & syscalls
Update 1	Implement <code>getprocinfo()</code> syscall	Add priority field; modify scheduler
Update 2	Build <code>cpuusage</code> user tool	Implement <code>setpriority()/getpriority()</code>
Final Report	Testing & documentation	Testing & documentation

5. Division of Work

- Akib Hassan Rifat (2312379042): Feature 1 `proc` struct fields, scheduler hooks, `getprocinfo()` syscall, `cpuusage` tool.
- Tauhid Ara Himu (2312187042): Feature 2 priority field, scheduler modification, `setpriority()` and `getpriority()` syscalls, test programs.

Both members will collaborate on final testing, the report, and the demonstration.

6. References

- xv6-riscv source: <https://github.com/mit-pdos/xv6-riscv>
- xv6 Book (RISC-V rev5): <https://pdos.csail.mit.edu/6.828/2025/xv6/book-riscv-rev5.pdf>
- OSTEP Projects: <https://github.com/remzi-arpacidusseau/ostep-projects>
- IIT Bombay xv6 guide: <https://www.cse.iitb.ac.in/~mythili/os/xv6.html>